TU Dortmund

Fachprojekt Detektion von Sicherheitslücken

M.Sc. Johannes Düsing, M.Sc. Anemone Kampkötter

JProf. Dr.-Ing. Ben Hermann

Aufgabe 5: Abstrakte Interpretation

Ausgabe: 06.12.2023

Besprechung: 20.12.2023

Eine sinnvolle Definition dafür, welcher Code innerhalb eines Softwaresystems (un-)erreichbar ist, ist schon lange Gegenstand aktueller Forschung und hat auch bei uns im Fachprojekt schon mehrfach eine Rolle gespielt. Aufgrund des *Satzes von Rice* ist eine *vollständig präzise* Antwort auf diese Frage im Allgemeinen nicht berechenbar, man kann sich jedoch auf verschiedenen Ebenen einer Lösung annähern. Eine gängige Überapproximation nutzt Callgraphen, um unerreichbare Methoden zu identifizieren.

Warum auch dies nur eine Überapproximation ist, zeigt das folgende Beispiel. Für uns Lesende zeigt sich schnell, dass das `if`-Statement in Zeile 5 niemals ausgeführt werden kann. Damit sind auch die Methoden-Aufrufe in Zeilen 6 und 7 toter Code, und können niemals ausgeführt werden. Trotzdem werden alle gängigen Callgraph-Algorithmen diese Aufrufe als *möglich* klassifizieren.

```
1 var i = 0
2
3 if(Math.random() > 0.5) i = 2
4
5 if(i == 1){
6   println("Integer entspricht dem Wert eins")
7   System.exit(1)
8 }
```

Eine präzisere Approximation erlaubt die sogenannte *Abstrakte Interpretation*. Dabei wird der Programmcode nicht ausgeführt, aber statisch *interpretiert*. Für jede Variable berechnet die abstrakte Interpretation an jeder Instruktion einen *abstrakten* Wert. Anhand dieser abstrakten Werte kann (je nach deren Präzision) entschieden werden, ob bestimmte Verzweigungen ausgeführt werden können oder eben nicht.

Die Art der abstrakten Werte wird über eine bestimmte mathematische Struktur, den sogenannten *Verband* (engl. *Lattice*) definiert. Ein einfaches Beispiel für die abstrakte Repräsentation von `Integer`-Werten sind *Intervalle*. In Zeile 1 gilt offensichtlich $i \in [0, 0]$, also $i = 0$. In Zeile 3 kann statisch nicht entschieden werden, ob das `if` ausgeführt wird oder nicht - die abstrakte Interpretation muss beides betrachten. Aus dem Fakt $i = 0$ vorher und dem neuen Fakt $i = 2$ wird also $i \in [0, 2]$, eine sichere Überapproximation. In Zeile 5 muss nun entschieden werden, ob das `if` ausgewertet werden muss oder nicht. Die abstrakte Interpretation weiß, dass $i \in [0, 2]$, damit könnte $i == 1$ gelten, also muss das `if` ausgewertet werden. Dies bringt also noch nicht den gewünschten Präzisions-Vorteil.

Besser wird es, wenn statt Intervallen *Mengen* (*Sets*) zur Repräsentation von Integer-Werten genutzt werden. Dann gilt in Zeile 1 $i \in \{0\}$, und in Zeile 3 $i \in \{0, 2\}$. Mit diesem Wissen kann in Zeile 5 eindeutig entschieden werden, dass das `if` nicht ausgewertet werden muss. Der Nachteil ist, dass die Mengen beliebig groß werden können und damit viel mehr Speicher brauchen als Intervalle, die immer nur aus zwei Zahlen bestehen.

So ergibt sich über die gewählte abstrakte Repräsentation (die *Domain*) ein Trade-Off zwischen Aufwand und Präzision.

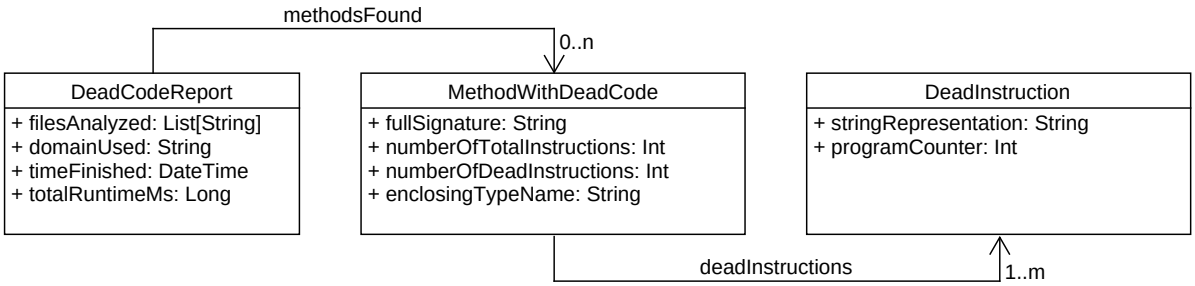

Abbildung 1: Ausgabe-Format als UML Klassendiagramm

5.1 Analyseaufgabe

Für diese Analyseaufgabe sollen Sie eine abstrakte Interpretation mittels OPAL umsetzen. Die Interpretation soll für jede Methode eines Softwaresystems einzeln gestartet werden und jeweils mögliche unerreichbare Instruktionen detektieren. Orientieren können Sie sich dafür an der Datei `AiDemo.scala`, die zeigt wie die abstrakte Interpretation für alle Methoden eines Systems aufgerufen werden kann, und wie die Präsenz von unerreichbaren Instruktionen detektiert werden kann.

Als Eingabe soll Ihre Analyse die Datei(en) des zu untersuchenden Softwaresystems akzeptieren. Zusätzlich soll wählbar sein, welche *Domain* für die Interpretation genutzt werden soll (siehe `AiDemo.scala`). Ausgegeben werden soll ein Bericht, der alle Methode in denen mindestens eine unerreichbare Instruktion vorhanden ist enthält, inklusive der genauen String-Repräsentation der Instruktionen. Der Bericht soll im JSON-Format abgespeichert werden. Semantisch muss der Bericht der in Figure 1 Struktur *zwingend entsprechend*, wie auch in Listing 1 beispielhaft gezeigt.

```
{
  "filesAnalyzed": [ "foo.jar", "bar.jar"],
  "domainUsed": "...",
  <...>
  "methodsFound": [
    {
      "fullSignature": "exampleSignature",
      <...>
      "deadInstructions": [
        {
          "stringRepresentation": "INVOKE_STATIC ...",
          "programCounter": 14
        },
        <...>
      ]
    },
    <...>
  ]
}
```

Listing 1: Beispiel für den Ausgabe-Bericht

5.2 Usability

Legen Sie bei der Umsetzung ihrer Analyse besonderen Wert auf die Usability der Anwendung. Das betrifft sowohl die *Eingaben* (wie einfach lässt sich die Analyse ausführen / konfigurieren?) als **besonders auch** die Ausgaben - machen Sie sich ausführliche Gedanken dazu, wie die Ergebnisse Ihrer Analyse den Nutzenden möglichst gut nähergebracht werden können, sodass diese nicht überfordert werde, aber auch alle nötigen Details (bei Bedarf) einsehen können. Erklären Sie in Ihrer Präsentation für dieses Aufgabenblatt ausführlich, warum Sie sich für ihre jeweilige

Lösung entschieden haben, und warum Sie der Meinung sind dass diese Lösung angemessen ist. Einige Ideen könnten sein:

- Ein UI dass die Ausgabe in geeigneter Form visualisiert (basierend auf dem JSON-Report)
- Python (oder R / etc..) Skripts, die aus dem JSON-Report hilfreiche Diagramme erzeugen
- Detaillierte (möglicherweise interaktive) Konsolen-Applikationen, die Einblicke in den JSON-Report ermöglichen

Hinweis: Da alle Gruppen einen JSON-Report erstellen, der dem gleichen Format folgt, sollten solche Tools prinzipiell auch auf den Ergebnissen von allen anderen Gruppen funktionieren!

5.3 Evaluationsaufgabe

Koordinieren Sie sich mit *allen anderen Gruppen* des Fachprojekts, und einigen Sie sich auf einen Benchmark aus 15 Java-Programmen (ohne TPLs). Diesen einheitlichen Benchmark sollen **alle Gruppen** nutzen, um die jeweils eigene Implementierung zu testen. Führen Sie ihre Analyse für jedes der 15 Programme jeweils mit mindestens vier verschiedenen Domains aus. Das sollte also zu mindestens $15 \times 4 = 60$ Ausgabe-Berichten führen, die Sie alle aufbewahren und in Ihr Repo mitaufnehmen sollen.

Werten Sie diese Daten anschließend **sinnvoll** aus. Interessant könnten die Unterschiede in den Laufzeiten und / oder Ergebnissen bei den unterschiedlichen Domains sein, aber auch die Laufzeitunterschiede bei gleichen Domains über verschiedene Projekte, sowie Gesamtlaufzeiten. Nutzen Sie zur Präsentation Ihrer Ergebnisse *Diagramme*.

Weitere Anforderungen:

- Stellen Sie sicher, dass Ihre Implementierung die geforderte Funktionalität korrekt umsetzt. Schreiben Sie mehrere Unit-Tests[1] für Ihren Codes und messen sie die Testabdeckung (Coverage). Legen Sie dabei besonderen Fokus auf die Korrektheit des Ausgabe-Formats.
- Stellen Sie uns Ihre Lösung wieder in Ihrem Git-Repository zur Verfügung.

[1] `https://www.baeldung.com/scala/scalatest`